



Figure 4: The UDP header

(Source: RFC 768)

```

aasisvinayak@GNU-BOK:~$ cat /proc/net/route

```

Interface	Destination	Gateway	Flags	RefCnt	Use	Metric	Mask	#
eth0	00000000	00000000	0001	0	0	1	00ffffff	0
eth0	0000f000	00000000	0001	0	0	1000	0000ffff	0
eth0	00000000	01000000	0003	0	0	0	00000000	0

Figure 5: The `/proc/net/route` file

```
aasisvinayak@GNU-BOK:~$ netstat -r
```

Kernel IP routing table	Destination	Gateway	Genmask	Flags	MSI	Window	irtt	Iface
	131.227.156.0	*	255.255.255.0	U	0 0	0	0	eth0
	link-local	*	255.255.0.0	U	0 0	0	0	eth0
	Default	resnet-156.0.0.0.0.0.0		UU	0 0	0	0	eth0

```
aasisvinayak@GNU-BOK:~$
```

Figure 6: The output for `netstat -r`

```

inet addr:127.0.0.1 Mask:255.0.0.0
inet6 addr: ::1/128 Scope:Host
UP LOOPBACK RUNNING MTU:16436 Metric:1
RX packets:974 errors:0 dropped:0 overruns:0 frame:0
TX packets:974 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:0
RX bytes:153795 (153.7 KB) TX bytes:153795 (153.7 KB)

pan0 Link encap:Ethernet HWaddr 66:a2:20:ba:f3:10
BROADCAST MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:0
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

vboxnet0 Link encap:Ethernet HWaddr 0a:00:27:00:00:00
BROADCAST MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

```

You may use the `down` option to shut down an interface, and `arp` to enable (or disable) the use of the ARP protocol on an interface. `io_addr addr` is another option to start an address in the I/O space. And if you

wish to assign an IP address, you may just employ the `address` option.

Displaying the output (content) of `/proc/interrupts` and analysing it is quite handy in some cases (say, if you want to look at the number of interrupts per IRQ). You can do this by issuing the following commands:

```

aasisvinayak@GNU-BOK:~$ sudo cat /proc/interrupts
[sudo] password for aasisvinayak:

```

CPU0	CPU1				
0:	2275331	1552933	IO-APIC-edge	timer	
1:	917	457	IO-APIC-edge	rtc0	
8:	0	1	IO-APIC-edge	rtc0	
9:	9	4	IO-APIC-fastio	acpi	
12:	79030	76412	IO-APIC-edge	rtc0	
17:	4193	2697	IO-APIC-fastio	eth2	
20:	457	159	IO-APIC-fastio	ehci_hcd:usb2, uhci_hcd:usb3, uhci_hcd:usb6	
21:	83736	41592	IO-APIC-fastio	uhci_hcd:usb4, uhci_hcd:usb7, HDA Intel	
22:	27	13	IO-APIC-fastio	ehci_hcd:usb1, uhci_hcd:usb5, uhci_hcd:usb8	
28:	79298	55301	PCI-MSI-edge	ahci	
29:	290499	306516	PCI-MSI-edge	eth0	
30:	181527	188303	PCI-MSI-edge	i915@pci:0000:00:02.0	
NMI:	0	0	Non-maskable interrupts		
LOC:	1551082	2026604	Local timer interrupts		
SPU:	0	0	Spurious interrupts		
CNT:	0	0	Performance counter interrupts		
PND:	0	0	Performance pending work		
RES:	789579	767812	Rescheduling interrupts		
CAL:	156	160	Function call interrupts		
TLB:	5216	7330	TLB shootdowns		
TRM:	0	0	Thermal event interrupts		
THR:	0	0	Threshold APIC interrupts		
MCE:	0	0	Machine check exceptions		
MCP:	34	34	Machine check polls		
ERR:	0				
MIS:					

If you wish to look at the static routing table, you can `cat` the `/proc/net/route` file. Figure 5 shows the output on my PC.

You might have tried the `netstat` command as well. But the above commands will work even if you don't have the `netstat` utility. (I have seen some changes in 2.4 and 2.6, but you need not worry about this now.)

Since most of the *distros* have this utility, you can issue `netstat -r` (or `-nr`) to obtain the routing table—readability is better in this case (refer to Figure 6). You can also use `route -n` for the same purpose.

`/etc/services` is a very vital file that helps you find how the port numbers are linked to the named services. For your reference, here is a standard entry in the file:

```

tcpmux      1/tcp          # TCP port service multiplexer

```